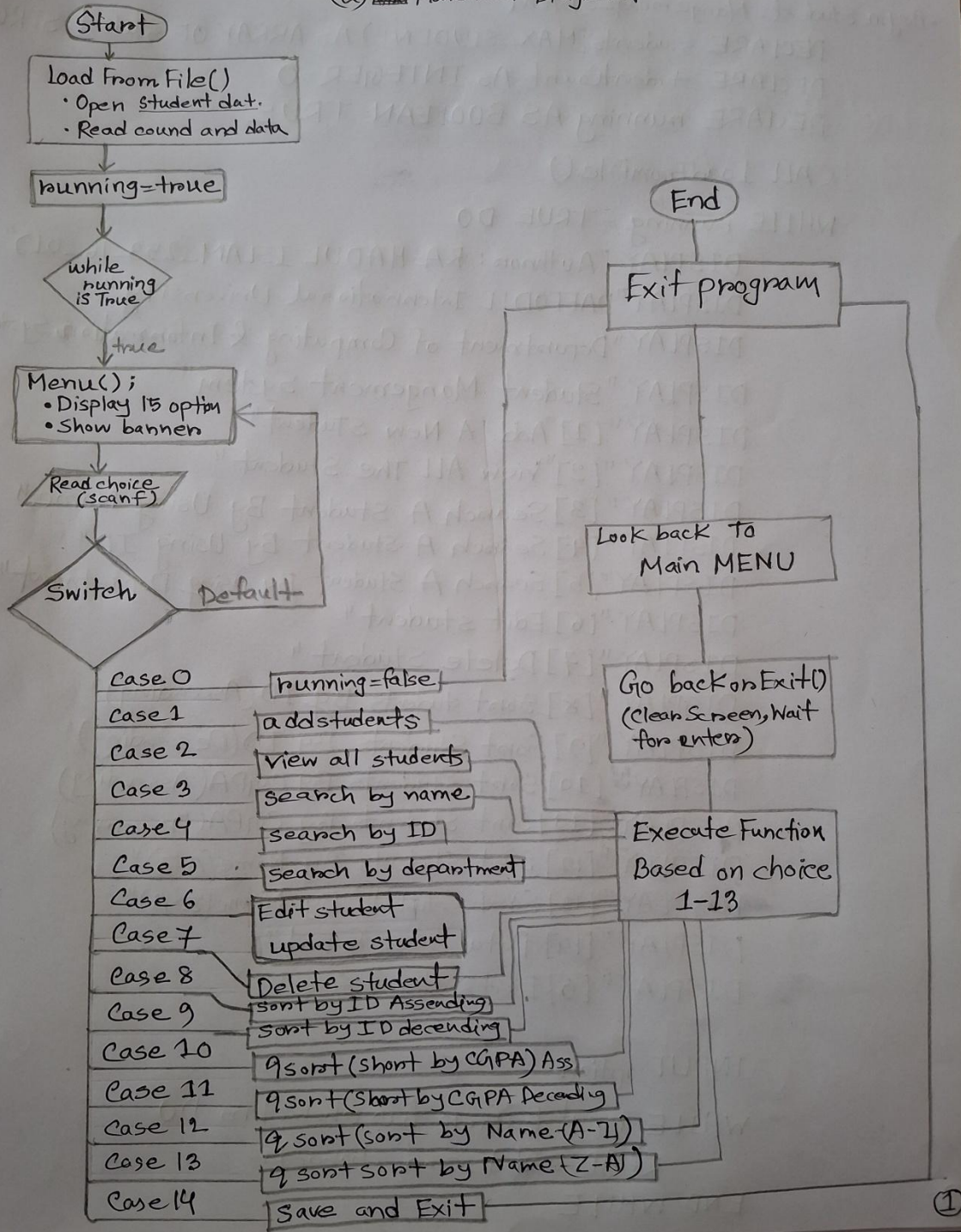


Student Management System Using C

"Project = Student Management System"

(a) Flowchart Diagram



(b) Pseudocode

• Begin Student-Management-System

DECLARE students[MAX_STUDENT) AS ARRAY OF student STRUCT

DECLARE studentCount AS INTEGER=0

DECLARE running AS BOOLEAN=TRUE

CALL LoadFromFile()

WHILE running = TRUE DO

DISPLAY "Authors: RASHADUL ISLAM-253-16-019"

DISPLAY "DAFFODIL International University"

DISPLAY "Department of Computing & Information System"

DISPLAY "Student Management System"

DISPLAY "[1] Add A New Student"

DISPLAY "[2] View All The Student"

DISPLAY "[3] Search A Student By Using Name"

DISPLAY "[4] Search A Student By Using ID"

DISPLAY "[5] Search A Student By using Department"

DISPLAY "[6] Edit student"

DISPLAY "[7] Delete Student"

DISPLAY "[8] Sort students By ID (Ascending)"

DISPLAY "[9] Sort Students By ID (Descending)"

DISPLAY "[10] Sort students By CGPA (Ascending)"

DISPLAY "[11] Sort Students By CGPA (Descending)"

DISPLAY "[12] Sort Students By Name (A-Z)"

DISPLAY "[13] Sort Students By Name (Z-A)"

DISPLAY "[14] Data Save & Exit"

DISPLAY "[0] Exit without Save"

INPUT option

WHILE input buffer has characters DO

READ and discard character

END WHILE

SWITCH option

Case 0:

running = FALSE
CALL exitProject()

Case 1:

CALL clearScreen()
CALL addStudent()
CALL goBackOnExit()

Case 2:

CALL clearScreen()
CALL viewAllStudents()
CALL goBackOnExit()

Case 4:

CALL clearScreen()
INPUT student_name
index = CALL searchByName(student_name)

IF index >= 0 THEN
 DISPLAY student details

Else IF

CALL goBackOnExit()

Case 4:

CALL clearScreen()
INPUT student_id
index = CALL searchById(student_ID)

IF index > 0 THEN
 DISPLAY student details

Else IF

CALL go Back on Exit()

// Similar Pattern for other cases

Case 14:

CALL saveToFile()

DISPLAY "Data saved successfully!"

running = FALSE

CALL exitProject()

```

    DEFAULT:
        DISPLAY "Invalid choice!"
        WAIT for Enter key
    END SWITCH
END WHILE
RETURN 0
END Student-Management-System
BEGIN addStudents
    IF studentCount >= Max_STUDENT THEN
        DISPLAY "Student limit reached!"
        RETURN
    END IF
    INPUT id
    IF CALL searchByID(id) != -1 THEN
        DISPLAY "STUDENT with this ID already exists!"
        RETURN
    END IF
    INPUT name
    INPUT department
    INPUT cgpa
    students[studentCount].id = id
    COPY name TO students[studentCount].name
    COPY department TO students[studentCount].department
    students[studentCount].cgpa = cgpa
    studentCount = studentCount + 1
    DISPLAY "Student Added Successfully!"
END addStudents

```

```
BEGIN searchByName(student_name)
```

```
  For i=0 TO studentCount-1 DO
```

```
    IF Case-Insensitive-Compare(students[i].name, student_name)
```

```
      THEN RETURN i
```

```
    END IF
```

```
  END FOR
```

```
  RETURN -1
```

```
END searchByName
```

```
// same patterns for other functions
```

```
// Function: sorting
```

```
BEGIN sortByIdAsc
```

```
  CALL qsort(students, studentCount, sizeof(student), compareBy  
                                                    IDAsc)
```

```
  CALL viewAllStudents()
```

```
END sortByIdAsc
```

```
// same patterns for other function
```

————— X —————

Task 2

(a) Data Structure Choice & Justification:

• Choice: Array of structure with static Allocation

• Justification:

(1) Fixed-Size Array Design:

```
#define MAX_STUDENT 1000
```

```
student students[MAX_STUDENT];
```

• Predictable Memory Usage: Always allocates space for 1000 students.

• No Runtime Allocation: Avoids memory fragmentation

• Simple Memory Management: OS handles allocation at program

(2) Efficient for Department Scale:

• Computing department typically has 500-800 students

• MAX_STUDENTS = 1000 provides comfortable buffers.

• No need for dynamic allocation overhead

(3) Struct Design Optimization:

```
typedef struct {  
    char name[30];  
    int id;  
    char department[50];  
    float gpa;  
} student;
```

• Memory Per student: $(30 + 4 + 50 + 4) = 88$ bytes (+ padding)

• Total Memory: $1000 \times 92 \text{ bytes} \approx 92 \text{ Kb}$ (efficient)

• Cache Friendly: Contiguous memory improves performance.

(4) File Handling Efficiency:

```
fwrite(students, sizeof(student), studentCount, file);
```

• Single write/read operation for entire array

• Binary format preserves data exactly

• Fast loading/saving even with 1000 students

(6)

(5) qsort() Compatibility:

- qsort(students, studentCount, sizeof(student), compareFunction)
- Arrays work perfectly with C's built-in qsort
 - Comparison functions are simple and efficient
 - Sorting operations are $O(n \log n)$

(6) Simple Index-Based Operation:

- $O(1)$ access for known indices
- Easy iteration with for loops
- Straight forward deletion by shifting

(B) Project Plan Based on My Implementation:

Phase 1: Foundation (2 Days)

Day 1:

- ✓ Define student structure with 4 fields
- ✓ Implement main menu with 14 options
- ✓ Create basic input/output functions
- ✓ Add author information (Rashadul Islam-253-16-019)

Day 2:

- ✓ Implement file handling (loadFromFile, saveToFile)
- ✓ Add input validation and buffers clearing
- ✓ Create clearScreen() for cross-platform compatibility
- ✓ Implement basic error handling

Phase 2: Core Features (3 Days)

Day 3: CRUD Operation

- ✓ Complete addStudents() with duplicate ID check
- ✓ Implement viewAllStudents() with formatted table
- ✓ Add search function(name, ID, department)

Day 4: Data Modification

- ✓ Implement updateStudent() for editing records
- ✓ Create deleteStudent() with confirmation
- ✓ Add case-insensitive search using strcasecmp

Day 5: User Interface

- ✓ Polish menu display with university branding
- ✓ Implement goBack or Exit() for navigation
- ✓ Add proper formatting and user prompts

Phase 3: Advanced Features (2 Days)

Day 6: Sorting Implementation

- ✓ Create 6 comparison functions for qsort
- ✓ Implement sortByIDAsc() and sortByDes()
- ✓ Add CGPA and name sorting options

Day 7: Testing and Polish

- ✓ Test all 14 features thoroughly
- ✓ Fix bugs and edge cases
- ✓ Optimize code structure and add comments
- ✓ Prepare for submission

Development Approach:

- Data Safety: Automatic Saving, confirmation for destructive action.
- User-friendly: Clear prompts, formatted output, error messages.
- Module Design: Each function has single responsibility.
- Incremental Development: Build and test each feature.

Tools Used:

- Compiler: GCC/MinGW
- Editor: VS Code/Code-Blocks
- Debugging: Print statements and systematic testing

⇒ (c) ^{Here} Write a pseudocode that Calculate and display the average CGPA of all students stored in the system.

⇒ FUNCTION Calculate Average CGPA

INPUT: student-list, total-students

IF total-students = 0 THEN

PRINT "NO students available to calculate average"

ELSE

SET Sum = 0

FOR i = 1 TO total-students

Sum = Sum + student-list[i].cgpa

END FOR

SET Average = Sum / total-students

PRINT "Average CGPA = ", average

END IF

END FUNCTION

_____ x _____

⇒ (d) Memory Allocation Strategy:

1. Static Compile-Time Allocation

students students[MAX-STUDENT];

- Size: $1000 \times \text{size of (student)} \approx 92,000$ bytes
- Location: Data segment of Program memory
- Lifetime: Entire program execution

2. Memory Layout of Your Struct:

name[30] → 30 bytes

id → 4 bytes

department[50] → 50 bytes

cgpa → 4 bytes

padding → ~4 bytes

(9)

* Advantages of Your Approach:

1. No Memory Leaks:

- Since you don't use dynamic allocation, no risk of memory leaks
- All memory freed automatically when program exits.

2. Predictable Performance:

- fixed memory footprint
- No allocation overhead during runtime
- Consistent performance regardless of student count

3. Simple Implementation:

- No complex pointers management
- Easy to understand and debug
- Less error-prone for beginners

□ Limitations:

1. Fixed Size Limitation:

```
#define MAX-STUDENT 1000
if (studentCount >= MAX-STUDENT){
    printf("InStudent limit reached!\n");
    return;
}
```

- Problem: Cannot exceed 1000 students
- Solution: Clear error message when limit reached
- Alternative: Could use dynamic array if needed

2. Memory Wastage:

- If only 100 students: $900 \times 92 \text{ bytes} \approx 82.8 \text{ Kb}$ wasted
- Trade-off for simplicity and performance.

3. No Dynamic Resizing:

- Cannot adjust size based on actual needs.
- Requires recompilation to change MAX-STUDENT

Task-2

(a) Lesson Learned and Future Improvements

• What I learned from Developing This Project:

1. Practical C programming Skills:

- File Handling: Binary file I/O with fread()/fwrite()
- Structure Implement: Using typedef struct for data organization
- Array Management: Handling fixed-size arrays with boundary checks
- String Operations: Case-insensitive comparison with strcasecmp()

2. System Design Principles:

- Separating concerns into functions
- Creative intuitive menus and prompts.
- Input validation and graceful error messages.
- Ensuring data survives program restarts.

3. Algorithm Implementation:

- Using qsort() with custom comparison functions.
- Linear search with case-insensitive options.
- Minimizing unnecessary computations.

4. Cross Platform Development:

- Learned to write a portable code for Windows and Linux
- Used preprocessor directives for platform-specific code.
- Implemented clearscreen() that works on both system

Possible Improvements for future Versions

1. Add showStatistics() function
2. Add CGPA validation (0.0-4.0)
3. Add confirmation for exit without save
4. Add backup file creation
5. Implement Dynamic Array
6. Add password protection
7. Add search history
8. Add CSV export/import
9. Implement binary search for ID lookup
10. Database integration (SQLite)